

A Size-Invariant Deep Reinforcement Learning Policy for the Interval Job Shop Scheduling Problem

Hernán Díaz [TODO: co-authors]

Draft v2 — July 2026

Abstract

The Interval Job Shop Scheduling Problem (IJSP) models uncertainty in task durations by closed intervals and asks for operation orderings that are robust under interval arithmetic. The state of the art is dominated by per-instance metaheuristic search. We present, to the best of our knowledge, the first learned *constructive* policy for the IJSP: a deep reinforcement learning (DRL) agent that dispatches operations one at a time and whose policy and value networks are, by construction, *independent of the instance size*. Size invariance is obtained by a dimensionless, uncertainty-aware feature design—all temporal quantities are normalized by an instance lower bound and interval widths enter as relative magnitudes—combined with a Deep Sets encoder over the set of eligible operations and a pooled global context. Trained with proximal policy optimization on only four 20×15 Taillard IJSP instances, the policy attains a mean relative error of 13.4% (12.3% best of three seeds) against crisp Taillard lower bounds on unseen 20×15 instances, versus $\approx 46\%$ for the best priority dispatching rule, constructing solutions in milliseconds. The same single network, evaluated zero-shot, outperforms the best dispatching rule on every tested instance of all seven Taillard size classes (15×15 to 50×20). A pre-registered, anchor-verified experimental methodology supports three further findings of practical relevance: sampling-based best-of- N inference yields consistent gains; self-attention over eligible operations does not improve on pooling at practical training budgets; and a single-size specialist generalizes across sizes better than multi-size training, even at an equalized per-instance budget. We additionally characterize the policy as an initializer for state-of-the-art tabu search populations, whose median sample dominates the best solution of randomized dispatching generators, and lay out a pre-registered hybridization study.

Keywords: job shop scheduling; interval uncertainty; deep reinforcement learning; neural combinatorial optimization; size invariance; hybrid metaheuristics.

1 Introduction

The Job Shop Scheduling Problem (JSP) is among the most studied NP-hard combinatorial optimization problems [1], both for its practical relevance in manufacturing and for its notorious difficulty—small instances proposed in the 1960s remained open for decades. In real production environments, the durations of operations are rarely known with precision at the time scheduling decisions must be made: tool wear, human factors, and material variability all introduce uncertainty. Among the frameworks proposed to capture this uncertainty—stochastic, fuzzy [2] and robust formulations—the *interval* representation stands out for requiring the least information from the decision maker: each duration is only assumed to lie within a closed interval, with no distributional or membership assumptions [4, 6]. The resulting Interval JSP (IJSP) evaluates a schedule with interval arithmetic, so its makespan is itself an interval, and solutions are compared through a ranking criterion such as the worst case or the expected (midpoint) makespan [6].

The current state of the art for makespan minimization in the IJSP is per-instance metaheuristic search: artificial bee colony algorithms [5, 6] and irace-tuned tabu search with specialized neighbourhoods [7] obtain mean relative errors (RE, against the crisp Taillard lower bounds) between roughly 0% and 14% depending on the size class, at a computational cost of seconds to minutes of search *per instance and per run*.

A complementary paradigm has emerged in the deterministic scheduling literature: *learning to dispatch*, where a deep reinforcement learning (DRL) agent constructs a schedule in a single pass by repeatedly choosing the next operation among the eligible ones [13, 14, 15]. Once trained, such a policy amortizes its cost over arbitrarily many instances and produces solutions in milliseconds, making it attractive (a) as a fast standalone solver in time-critical or embedded settings, and (b) as a high-quality initializer for search-based methods. To our knowledge, no learned constructive policy had been proposed or evaluated for the interval variant of the JSP.

Bringing DRL to the IJSP raises two specific challenges. First, *uncertainty must enter the state representation*: the agent should distinguish an operation with duration [10, 11] from one with [5, 16], since a min-max (or expected-value) objective values them very differently. Second, and more structurally, standard JSP-DRL architectures tie network dimensions to the instance size (e.g. value functions over fixed-size state vectors), so a policy trained on one size class cannot even be *evaluated* on another. This is at odds with the practical promise of learned solvers, which is precisely to amortize training across a distribution of instances.

Contributions. This work makes four contributions:

1. **A size-invariant, uncertainty-aware DRL policy for the IJSP** (Section 4). Every feature is dimensionless: temporal quantities are normalized by a per-instance lower bound, and interval uncertainty enters as relative widths. A Deep Sets [16] encoder over the eligible-operation set, combined with a pooled global context, yields policy *and* value networks with $\approx 1.2 \times 10^5$ parameters independent of the numbers of jobs and machines.
2. **A rigorous, pre-registered experimental methodology** (Section 5): a two-stage acceptance protocol (cheap filter benchmark, full confirmation benchmark) with paired seeds, bit-reproducible runs, and *deterministic anchors*—dispatching rules whose results must remain identical across all code versions, ensuring that measured improvements are attributable to learning and not to inadvertent environment changes.
3. **An empirical characterization** (Section 6): training-budget scaling on the training size class; zero-shot transfer that dominates the best dispatching rule on all seven Taillard size classes from a single network; and three controlled ablations (best-of- N inference, attention versus pooling, specialist versus multi-size training), two of which are cautionary negative results for the field.
4. **A bridge to hybrid search** (Section 7): a characterization of the policy as a generator of initial populations for the tabu search of [7], together with a pre-registered pilot design measuring anytime convergence under different seeding fractions.

2 Related Work

2.1 Job shop scheduling under interval uncertainty

Scheduling under uncertainty has been addressed through stochastic programming, fuzzy sets and interval analysis. Fuzzy processing times, usually triangular, have received sustained attention in the JSP [2, 3]; the interval representation can be seen as its least-committal special case, requiring only support bounds. Lei [4] introduced population-based neighbourhood search for the JSP with interval processing times; subsequent work developed dedicated interval ranking operators and robustness analyses. The recent IJSP literature centred on the Taillard-derived benchmark [5, 6, 7] establishes artificial bee colony variants and tabu search as the strongest published solvers, and fixes the evaluation conventions adopted here (interval arithmetic, lexicographic ranking during search, expected makespan for reporting). [TODO: ampliar con 3–5 referencias recientes de IJSP/robust scheduling que Hernán conoce mejor]

2.2 Neural combinatorial optimization and DRL for scheduling

Neural approaches to combinatorial optimization began with pointer networks [9] and their RL training [10], with attention-based encoders becoming standard for routing [12, 11]. For the deterministic JSP, Zhang et al. [13] learn dispatching policies over disjunctive-graph GNN embeddings; Park et al. [14] refine GNN representations and report generalization across sizes; Tassel et al. [15] provide an efficient RL environment formulation. Sampling-based decoding (best-of- N) is a standard inference-time improvement [12]. Permutation-invariant set encoders were formalized as Deep Sets [16]; attention is their natural strict generalization. Our architecture deliberately starts from the simpler Deep Sets construction and evaluates attention as a controlled ablation—a comparison that, at our budgets, favours the simpler model.

2.3 Hybrid learning and search

Combining learned constructive policies with local search follows the long-standing observation that metaheuristic performance depends on initialization quality and diversity. GRASP formalizes randomized greedy construction followed by local search [18]; in neural combinatorial optimization, learned constructions have been combined with beam search, sampling and improvement operators [12, 13]. We contribute an initialization study specific to the IJSP, with the population-based tabu search of [7] as the host solver and randomized dispatching as the fair baseline generator.

3 Problem Statement

Definition 1 (IJSP). *An IJSP instance comprises n jobs $J_1, \dots, J_n$ and m machines. Job J_j is a fixed sequence of m operations $(o_{j1}, \dots, o_{jm})$, where o_{jk} requires a specific machine ν_{jk} and an uncertain processing time known to lie in the interval $\mathbf{p}_{jk} = [p_{jk}^L, p_{jk}^U]$, $0 \leq p_{jk}^L \leq p_{jk}^U$. Each machine processes at most one operation at a time and preemption is not allowed.*

A solution is a processing order of all nm operations compatible with job precedences; we encode it as a *permutation with repetition* of job indices, where the k -th occurrence of j denotes o_{jk} . Given an order, its *semiactive schedule* starts every operation as early as its job and machine predecessors permit. Under interval arithmetic, with component-wise addition $[a, b] + [c, d] = [a + c, b + d]$ and join $\max([a, b], [c, d]) = [\max(a, c), \max(b, d)]$, starts and completions are intervals:

$$\mathbf{s}_o = \max(\mathbf{c}_{JP(o)}, \mathbf{c}_{MP(o)}), \quad \mathbf{c}_o = \mathbf{s}_o + \mathbf{p}_o, \quad (1)$$

where $JP(o)$ and $MP(o)$ are the job and machine predecessors of o . The interval makespan $\mathbf{C}_{\max} = [C_{\max}^L, C_{\max}^U]$ is the maximum of job completions, taken lexicographically by upper bound and, on ties, lower bound—the ranking used throughout search, corresponding to min-max robust optimization. For reporting we follow the field convention [6, 7]:

$$\text{RE}(\%) = \frac{E[\mathbf{C}_{\max}] - \text{LB}}{\text{LB}} \times 100, \quad E[\mathbf{C}_{\max}] = \frac{1}{2}(C_{\max}^L + C_{\max}^U), \quad (2)$$

with LB the published lower bound of the crisp Taillard counterpart [8]—an indicative, hardware-independent yardstick rather than a strict optimality gap, since the crisp JSP and the IJSP are different problems.

Remark 1. *For instances whose intervals are symmetric perturbations of a crisp instance, the midpoint scenario coincides with the original crisp durations; comparisons under Eq. (2) are therefore scenario-consistent (midpoint solution quality against the midpoint-scenario bound).*

4 Method

4.1 Dispatching as a Markov decision process

We formulate schedule construction as a finite-horizon MDP of exactly nm steps. The *state* after t steps comprises the partial schedule (job pointers, job and machine completion intervals) and the set

Table 1: Per-operation features (all dimensionless). $d = \mathbf{p}_{jk}$, $\mathbf{est}$ = earliest start interval, $\mathbf{rem}$ = remaining work interval of the job after o ; $\text{mid}(\cdot)$ is the interval midpoint; $L(\mu)$ the remaining worst-case load of machine μ ; $\bar{L}$ its average over machines.

#	Feature	Definition
1–2	duration bounds	$d^L/\text{LB}, d^U/\text{LB}$
3	duration uncertainty	$(d^U - d^L)/\text{mid}(d)$
4–5	earliest start bounds	$\text{est}^L/\text{LB}, \text{est}^U/\text{LB}$
6	start uncertainty	$(\text{est}^U - \text{est}^L)/\text{mid}(\mathbf{est})$
7–8	remaining job work	$\text{rem}^L/\text{LB}, \text{rem}^U/\text{LB}$
9–10	job position	$(m - k - 1)/m, k/m$
11	machine load	$L(\mu)/\text{LB}$
12	potential idle gap	$\max(0, c_\mu^U - c_\mu^L)/\text{LB}$
13	machine congestion	$L(\mu)/\bar{L}$
14	slack	$(\text{est}^U - \min_{o' \in \mathcal{E}} \text{est}_{o'}^U)/\text{LB}$
15–16	completions	$c_\mu^U/\text{LB}, c_\mu^L/\text{LB}$

Table 2: Global context features (all dimensionless aggregates; no dimension depends on n or m).

Feature	Definition
progress	scheduled operations / nm
partial makespan	$\max_\mu c_\mu^U/\text{LB}$
machine completions	mean and std of c_μ^U/LB
remaining loads	mean, max and std of $L(\mu)/\text{LB}$
remaining job work	mean and max of job remaining / LB
eligibility	$ \mathcal{E} /n$
pending uncertainty	$\sum(\text{widths})/\sum(\text{midpoints})$ of unscheduled ops
total load	$\sum_\mu L(\mu)/(\text{LB} \cdot m)$

$\mathcal{E}_t$ of eligible operations—the next unscheduled operation of each unfinished job, so $1 \leq |\mathcal{E}_t| \leq n$. The *action* selects one element of $\mathcal{E}_t$; the transition applies Eq. (1). Episodes always terminate after nm steps.

The *reward* is a weighted sum of six shaping components, each collapsing intervals by their worst case (upper bound): a terminal makespan term (weight 1.0; negative scaled makespan plus a bonus within 5% of the instance bound), a dense local-improvement term (weight 0.3; the step change of the projected makespan, penalizing deteriorations twice as much as it rewards improvements), machine idleness (weight 0.15), machine load balance (weight 0.15), operation criticality (weight 0.05) and job progress (weight 0.05). The weights were tuned through the benchmark-gated protocol of Section 5.2; component scales adapt automatically to each instance through its lower bound.

4.2 Dimensionless, uncertainty-aware features

Let LB denote the instance lower bound (worst case if the bound is itself an interval), used as the *única* temporal scale. Each eligible operation $o = o_{jk}$ on machine $\mu = \nu_{jk}$ is described by the 16 dimensionless quantities of Table 1; the global context is the 12-dimensional aggregate of Table 2. Two design rules follow from ablation evidence (Section 6.5): (i) features must add information that is *not* a linear combination of others (interval widths enter as relative, hence non-linear, magnitudes); and (ii) normalization must be co-designed with initialization and learning rates rather than retrofitted.

4.3 Size-invariant architecture

Let $x_i \in \mathbb{R}^{16}$ be the features of eligible operation i and $g_0 \in \mathbb{R}^{12}$ the global features. A shared two-layer MLP ϕ (width $h=128$, LayerNorm, ReLU, orthogonal initialization) embeds each candidate,

Table 3: Hyperparameters (identical across all experiments).

optimizer	Adam ($3 \cdot 10^{-4}$, $\epsilon=10^{-5}$)	discount γ	1.0
PPO clip	0.2	GAE λ	0.95
epochs / minibatch	4 / 256	update period	4 episodes
entropy coef.	0.01	gradient clip	0.5
hidden width	128	greedy eval. period	10 episodes
advantage norm.	per update	initialization	orthogonal

Algorithm 1 Best-of- N inference

```

1:  $(\sigma^*, \mathbf{C}_{\max}^*) \leftarrow$  greedy rollout of  $\pi$ 
2: for  $i = 1$  to  $N - 1$  do
3:    $(\sigma, \mathbf{C}_{\max}) \leftarrow$  sampled rollout of  $\pi$ 
4:   if  $\mathbf{C}_{\max} \prec_{lex} \mathbf{C}_{\max}^*$  then  $(\sigma^*, \mathbf{C}_{\max}^*) \leftarrow (\sigma, \mathbf{C}_{\max})$ 
5:   end if
6: end for
7: return  $\sigma^*$ 

```

$\phi_i = \phi(x_i)$. The context vector aggregates the candidate set permutation-invariantly [16]:

$$g = \text{MLP}_{ctx}([\frac{1}{|\mathcal{E}|} \sum_i \phi_i; \max_i \phi_i; g_0]) \in \mathbb{R}^h. \quad (3)$$

The *policy head* scores each candidate and applies a masked softmax, and the *value head* reads the context alone:

$$\pi(i | s) = \text{softmax}_i \text{MLP}_{\pi}([\phi_i; g]), \quad V(s) = \text{MLP}_V(g). \quad (4)$$

No parameter dimension depends on n or m : the same $\approx 1.2 \times 10^5$ -parameter network dispatches any instance. The last policy layer is initialized with gain 10^{-2} so initial logits are near-uniform. Optionally, a stack of L pre-LN self-attention blocks (multi-head, 4 heads, no positional encoding, padding-masked) transforms $\{\phi_i\}$ before pooling; $L=0$ recovers the base model exactly, and Section 6.4 evaluates $L=2$.

During batched training, variable-size candidate sets are padded to the batch maximum with a boolean mask applied to attention, pooling and logits; we verify (unit tests) that masked-padded forward passes are numerically identical to unpadded ones.

4.4 Training and inference

We train with PPO [17]; Table 3 lists all hyperparameters. Two choices depart from common JSP-DRL practice and are enabled by co-design with the normalized representation: $\gamma = 1$ (undiscounted finite horizon—with $\gamma = 0.99$ the terminal makespan signal reaches early decisions attenuated by $0.99^{nm} \approx 0.05$ for 20×15 instances) and batched minibatch updates with advantage normalization. Training on a set of instances proceeds in blocks (one instance at a time) with full weight transfer; sampled training episodes are interleaved with periodic greedy evaluations, and the best solution/model is tracked across both.

Best-of- N inference. At evaluation time we exploit the stochastic policy: one greedy rollout plus $N - 1$ samples, keeping the best solution by worst-case makespan ($N=64$). This choice is motivated empirically in Section 6.3: during training, the best solutions frequently originate from sampled episodes, i.e. the policy is more valuable as a *distribution* than as its argmax—consistent with sampling-based decoding in neural combinatorial optimization [12].

5 Experimental Methodology

5.1 Instances, data split and metric

We use the interval Taillard benchmark TA1–TA70 (seven size classes, symmetric interval perturbations of the classic instances [8]; see [6, 7]). The final model is trained on **TA11–TA14** only (20×15); **TA15–TA20** constituted the *development set*: no gradients, but every configuration decision was selected on their performance, so we report them separately and exclude them from claims about unseen data. All other instances are entirely unseen by both weights and development. Three independent training seeds are reported as mean and best. Performance is measured by RE, Eq. (2), using the published crisp lower bounds.

5.2 Pre-registered, anchor-verified protocol

All development followed a fixed protocol. (i) *Two-stage acceptance*: any proposed change was first evaluated on a cheap filter benchmark (1 training instance, 30 episodes, 2 seeds) that can only *reject*; survivors ran the full benchmark (4 training instances, 100–1000 episodes, 3 seeds), whose pre-registered rule (mean improvement beyond a noise threshold, or strict per-instance dominance) alone could *accept*. (ii) *Paired seeds*: identical seeds across configurations enable paired comparisons. (iii) *Deterministic anchors*: SPT, LPT, MOR and MWKR results are recomputed in every run and verified identical across all code versions—any deviation indicates an inadvertent change of environment semantics and invalidates the comparison. (iv) *Bit-reproducibility*: rerunning a configuration with the same seed reproduces results exactly. Every accepted and rejected experiment, with its data, is versioned in the project repository. This methodology caught, among others, thirteen superficially plausible single-factor modifications whose filter or confirmation results were clearly negative.

5.3 Hyperparameter configuration

All reported results use the literature-standard hyperparameters of Table 3 *without problem-specific tuning*. To assess whether this leaves performance on the table, we ran an automatic configuration study with irace [20]—the same tool and methodology used to configure the tabu search of [7]—racing an 8-parameter space (learning rate, entropy coefficient, PPO clip, epochs, minibatch size, update period, GAE λ , hidden width) over a budget of 330 tuning experiments at reduced fidelity (1 training instance, 300 episodes, ~ 8 minutes per evaluation), with training seeds as racing instances. The five elite configurations agreed on a coherent faster-learning regime (learning rate $7\text{--}9 \cdot 10^{-4}$, clip 0.3, minibatch 128, updates every 2 episodes, $\lambda=0.90$) and were insensitive to network width. However, at the full 1000-episode operating budget the best elite scored 14.5% mean RE versus 13.4% for the defaults (+1.66% mean makespan; five of six development instances within noise, one significantly worse) at $\sim 60\%$ higher training cost. Two conclusions follow: the low-fidelity optimum does not transfer to the operating budget—a caution for tuning-on-a-budget practices in DRL—and the defaults sit on the achievable performance plateau, i.e. our results do not depend on fine tuning.

5.4 Baselines

Constructive baselines comprise priority dispatching rules (SPT, LPT, MOR, MWKR, with interval-aware lexicographic comparisons; MOR is uniformly the strongest rule) and the Giffler–Thompson active schedule generator [19] with rule tie-breaks inside the machine conflict set. G&T with MWKR tie-break is the strongest deterministic constructive baseline on this benchmark: 29.4% mean RE across the 70 instances, versus 45.4% for plain MOR dispatching (the SPT tie-break variant performs poorly, 70.6%). Published IJSP metaheuristics (fEABC [5], ESABC [6], TS [7]; 30 runs and minutes of per-instance search on dedicated hardware) are included as reference yardsticks from a different computational class rather than as direct competitors. For calibration across the full benchmark: best-of-1024 policy sampling attains 12.7% mean RE over the 70 instances—ahead

Table 4: RE (%) per development instance, mean [best] over 3 seeds, Deep Sets model, best-of-64 inference.

Budget	TA15	TA16	TA17	TA18	TA19	TA20
100 eps	34.1 [31.9]	26.8 [22.8]	28.1 [24.8]	26.2 [24.9]	28.9 [24.1]	24.0 [21.6]
300 eps	18.7 [17.1]	16.1 [15.4]	13.5 [12.3]	16.9 [16.4]	16.3 [15.5]	16.8 [16.1]
1000 eps	13.6 [12.1]	12.5 [11.8]	13.0 [11.8]	15.9 [14.6]	12.0 [11.6]	13.4 [12.0]
MOR	51.3	35.8	50.1	54.3	43.2	43.7

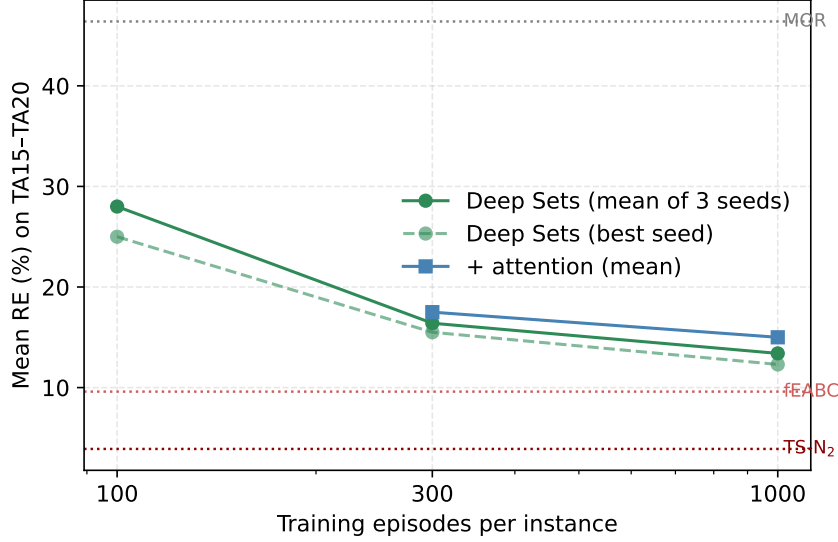


Figure 1: Mean RE on the development set versus training episodes per instance. Dotted lines: best dispatching rule and published metaheuristics (different computational class) as yardsticks.

of every deterministic constructive baseline on every instance, within 2.2–4.6 points of the fEABC 30-run average per class, and behind tuned tabu search everywhere.

6 Results

6.1 In-size performance and budget scaling

Table 4 reports per-instance RE on the development set for increasing training budgets, and Figure 1 the aggregate picture. Three observations: (i) the policy scales monotonically with budget (28.0% $\rightarrow$ 16.4% $\rightarrow$ 13.4% mean RE at 100/300/1000 episodes per instance) with clearly diminishing returns at the last step, locating the pure-budget ceiling near 11–12%; (ii) seed variance collapses with budget (per-instance makespan std of 10–24 units at 1000 episodes), indicating consistent learning rather than lucky sampling; (iii) the gap to the best dispatching rule is a factor ≈ 3.5 .

6.2 Zero-shot cross-size generalization

The size-invariant design allows direct evaluation of the 20×15 -trained network on any class. Figure 2 and Table 5 report zero-shot RE on unseen instances of all seven classes: the policy outperforms MOR on *every* tested instance, with graceful degradation (best classes: 7–10%; hardest transfers, 30×20 and 50×15 : 24–25%) and no failure mode up to 1000-operation episodes. Notably, transfer quality improves with in-size training budget (e.g. 30×20 : 32.3% with the 300-episode checkpoint versus 24.5% with the 1000-episode one), i.e. the specialist learns structure that is not size-bound.

Table 5: Zero-shot RE (%) of the 20×15 specialist (single network, best-of-64) versus MOR. †: 300-episode checkpoint.

Instance	v2	MOR	Instance	v2	MOR
TA1 (15×15)	14.4	29.7	TA31 (30×15)	13.9	34.4
TA2 (15×15)	7.0	34.8	TA41 (30×20)	24.5	53.5
TA5 (15×15)	8.7	50.7	TA51 (50×15) [†]	25.0	38.2
TA21 (20×20)	9.6	40.8	TA61 (50×20)	15.1	48.7
TA25 (20×20)	15.9	53.2			

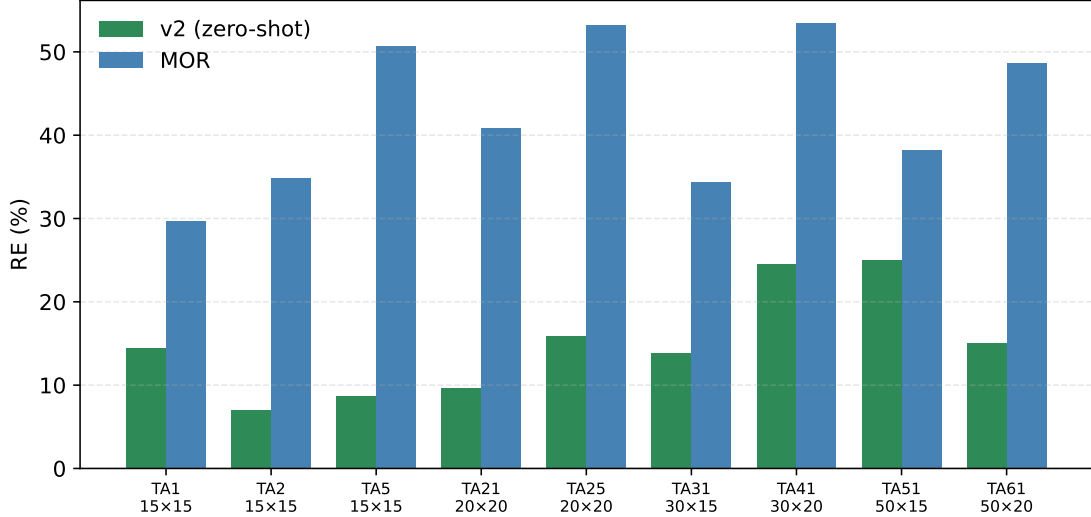


Figure 2: Zero-shot RE by instance across all seven Taillard size classes for the 20×15 -trained specialist versus MOR.

6.3 Ablation: best-of- N inference

Introducing best-of-64 inference (over one greedy rollout) improved the full-benchmark mean makespan by 3.6%, strictly better on 3 of 6 development instances and worse on none, while sharply reducing seed variance. Diagnostically, before this change the best training solutions frequently arose in early, exploratory episodes and were never surpassed by the greedy policy—evidence that the policy’s value concentrates in its sampling distribution.

6.4 Ablation: attention versus pooling

Two pre-LN self-attention blocks over the candidate embeddings (identical code path, one flag) were evaluated at both operating points with paired seeds: +1.2% mean makespan at 300 episodes (all six instances within the noise threshold) and +1.6% at 1000 (one instance significantly worse), at $\approx 30\%$ higher per-episode cost (Table 6). We conclude that at these budgets the pooled context already conveys the interaction information the policy can exploit; richer pairwise structure may require larger budgets or instance distributions to pay off. This negative result is reported with the same rigor as the positive ones.

6.5 Ablation: specialist versus multi-size training

Size invariance enables training on mixed size classes, a natural hypothesis for improving transfer. We trained agents on twelve instances spanning 15×15 , 20×15 and 20×20 , at matched total budget (3000 episodes) and at matched *per-instance* budget (12,000 episodes), against the 20×15 specialist. The specialist won on *all* five common evaluation instances in both settings—including instances of the

Table 6: Attention ablation: mean [best] RE (%) on the development set.

Configuration	300 eps	1000 eps
Deep Sets	16.4 [15.5]	13.4 [12.3]
+ attention ($L=2$)	17.5 [16.1]	15.0 [13.9]

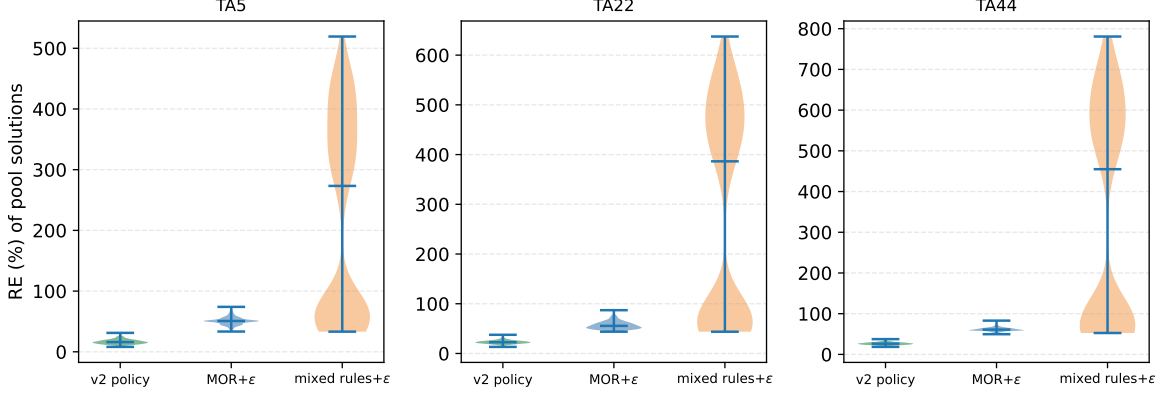


Figure 3: Seed-pool quality: RE distribution of 1024 sampled solutions per generator on the three pilot instances (violin plots, medians marked).

size classes the multi-size agent itself trained on (budget-equalized: 8.7 vs. 9.5 on 15×15 ; 15.9 vs. 18.5 on 20×20 ; 24.5 vs. 28.3 on the unseen 30×20). Per-instance training depth dominates size diversity, and—contrary to the intuition that transfer requires diverse training—specialist generalization is the stronger phenomenon. Earlier development iterations provide the complementary caution: appending features that are linear combinations of existing ones, or retrofitting input normalization onto a system co-adapted to raw scales, degraded performance by 9–113%; representation changes must be co-designed with optimization.

6.6 Positioning and computational cost

Training the final model takes ≈ 80 minutes per seed (4×1000 episodes) on a commodity CPU; constructing a solution takes milliseconds (greedy) to a few seconds (best-of-64 on 50×20). The policy’s standalone RE (13.4% in-size; 7–25% zero-shot) sits between dispatching rules (46%) and per-instance metaheuristics (4–10%)—establishing, for the IJSP, the same computational-class trade-off documented for the deterministic JSP [13], and motivating the hybrid use studied next.

7 Toward Hybridization: Seeding Tabu Search Populations

The tabu search of [7] initializes each run with a population of 250 individuals. We propose seeding a fraction p of the population with policy samples, and characterize first the raw material: for three pilot instances—clean for both methods (outside RL training/development and outside the TS irace tuning set)—we sampled 1024 solutions from the policy and from two randomized dispatching generators (MOR with ϵ -greedy noise, $\epsilon=0.1$; and a mixed-rule GRASP-style generator [18]). Figure 3 shows the RE distributions: the *median* policy sample is better than the *best* of 1024 heuristic individuals on all three instances, by 15–25 RE points (e.g. TA22: policy best/median 13.2%/22.9%; MOR- ϵ best 43.8%).

The pre-registered pilot (in execution) compares five arms—control; policy seeding at $p \in \{10, 50, 100\%\}$; MOR- ϵ seeding at $p=50\%$ —with 10 paired-seed runs per arm and instance, recording anytime incumbent curves. Pre-registered hypotheses: (H1) policy seeding reduces time-to-target (time to reach the control’s final RE); (H2) the benefit exceeds heuristic seeding at equal p , attributing

it to the generator rather than to any warm start; (H3) an intermediate p is optimal, as full seeding may reduce population diversity. [TODO: insertar resultados del piloto: curvas anytime, time-to-target, Wilcoxon pareado]

8 Conclusions and Future Work

We presented the first learned constructive policy for the interval job shop scheduling problem, built on a dimensionless, uncertainty-aware state representation that makes policy and value networks independent of the instance size. A single network trained on four instances of one size class dominates the best priority dispatching rule on the entire Taillard IJSP grid in zero-shot evaluation, and scales predictably with training budget on its own class. Our pre-registered, anchor-verified methodology additionally establishes two negative results we believe are valuable to the community: at practical budgets, self-attention over candidates does not improve on pooled context, and multi-size training does not improve on—indeed loses to—a single-size specialist, even budget-equalized.

The policy’s sampling distribution makes it a strong initializer for population-based search, dominating randomized dispatching generators by wide margins; the ongoing hybridization study quantifies the resulting anytime gains within a state-of-the-art tabu search. Beyond it, we see three directions: instance distributions with heterogeneous uncertainty, where min–max and expected-value criteria genuinely diverge; curricula and fine-tuning protocols that exploit size invariance; and tighter integrations where the policy guides neighbourhood construction rather than only initialization.

Reproducibility. Code, benchmark harness, all experiment records (accepted and rejected), final checkpoints and figure-generation scripts are available in the project repository. [TODO: URL/Zenodo DOI al publicar]

References

- [1] M. R. Garey, D. S. Johnson, R. Sethi. The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2):117–129, 1976.
- [2] I. González-Rodríguez, C. R. Vela, J. Puente. A genetic solution based on lexicographical goal programming for a multiobjective job shop with uncertainty. *Journal of Intelligent Manufacturing*, 21:65–73, 2010. [TODO: verificar/decidir referencia fuzzy JSP más adecuada]
- [3] J. J. Palacios, I. González-Rodríguez, C. R. Vela, J. Puente. Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems*, 278:81–97, 2015.
- [4] D. Lei. Population-based neighborhood search for job shop scheduling with interval processing time. *Computers & Industrial Engineering*, 62(4):1200–1208, 2012.
- [5] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing*, 22(4):645–657, 2023.
- [6] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering*, 30(3):223–242, 2023.
- [7] H. Díaz et al. Neighbourhood structures and ranking operators for the interval job shop scheduling problem. Under review, 2026.
- [8] E. Taillard. Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2):278–285, 1993.
- [9] O. Vinyals, M. Fortunato, N. Jaitly. Pointer networks. *NeurIPS*, 2015.

- [10] I. Bello, H. Pham, Q. V. Le, M. Norouzi, S. Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv:1611.09940*, 2016.
- [11] M. Nazari, A. Oroojlooy, L. Snyder, M. Takáč. Reinforcement learning for solving the vehicle routing problem. *NeurIPS*, 2018.
- [12] W. Kool, H. van Hoof, M. Welling. Attention, learn to solve routing problems! *ICLR*, 2019.
- [13] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, C. Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS*, 2020.
- [14] J. Park, J. Chun, S. H. Kim, Y. Kim, J. Park. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, 59(11):3360–3377, 2021.
- [15] P. Tassel, M. Gebser, K. Schekotihin. A reinforcement learning environment for job-shop scheduling. *arXiv:2104.03760*, 2021.
- [16] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, A. Smola. Deep sets. *NeurIPS*, 2017.
- [17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.
- [18] T. A. Feo, M. G. C. Resende. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6:109–133, 1995.
- [19] B. Giffler, G. L. Thompson. Algorithms for solving production-scheduling problems. *Operations Research*, 8(4):487–503, 1960.
- [20] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, T. Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.